Pseudo random number generators

Leonardo Toffalini

2026-02-26

Outline

1. Intuition 2
2. The classics 5
3. Formalism 17
4. One-way function candidates 25
5. Beyond random bits 29

1. Intuition

1.1 What we expect from a PRNG

1.1 What we expect from a PRNG

1. Given a short input (seed) it produces a long *seemingly random* sequence.

1.1 What we expect from a PRNG

1. Given a short input (seed) it produces a long *seemingly random* sequence.
2. Generation should be really fast.

1.1 What we expect from a PRNG

1. Given a short input (seed) it produces a long *seemingly random* sequence.
2. Generation should be really fast.
3. The sequence must be reproducible just from the seed.

1.1 What we expect from a PRNG

1. Given a short input (seed) it produces a long *seemingly random* sequence.
2. Generation should be really fast.
3. The sequence must be reproducible just from the seed.

The sad truth

- What does *seemingly random* mean?

1.1 What we expect from a PRNG

1. Given a short input (seed) it produces a long *seemingly random* sequence.
2. Generation should be really fast.
3. The sequence must be reproducible just from the seed.

The sad truth

- What does *seemingly random* mean?
- We cannot use Kolmogorov complexity to measure randomness, because that would not satisfy 1.

1.1 What we expect from a PRNG

1. Given a short input (seed) it produces a long *seemingly random* sequence.
2. Generation should be really fast.
3. The sequence must be reproducible just from the seed.

The sad truth

- What does *seemingly random* mean?
- We cannot use Kolmogorov complexity to measure randomness, because that would not satisfy 1.
- We cannot use physical methods like radioactive radiation as they would not satisfy 2. and 3.

1.2 Reconciliation

We need to redefine what *seemingly random* means.

1.2 Reconciliation

We need to redefine what *seemingly random* means.

Possible definitions:

1.2 Reconciliation

We need to redefine what *seemingly random* means.

Possible definitions:

"There must not exist a randomized polynomial time algorithm that can differentiate between truly random and generated sequences with *non-negligible advantage*."

1.2 Reconciliation

We need to redefine what *seemingly random* means.

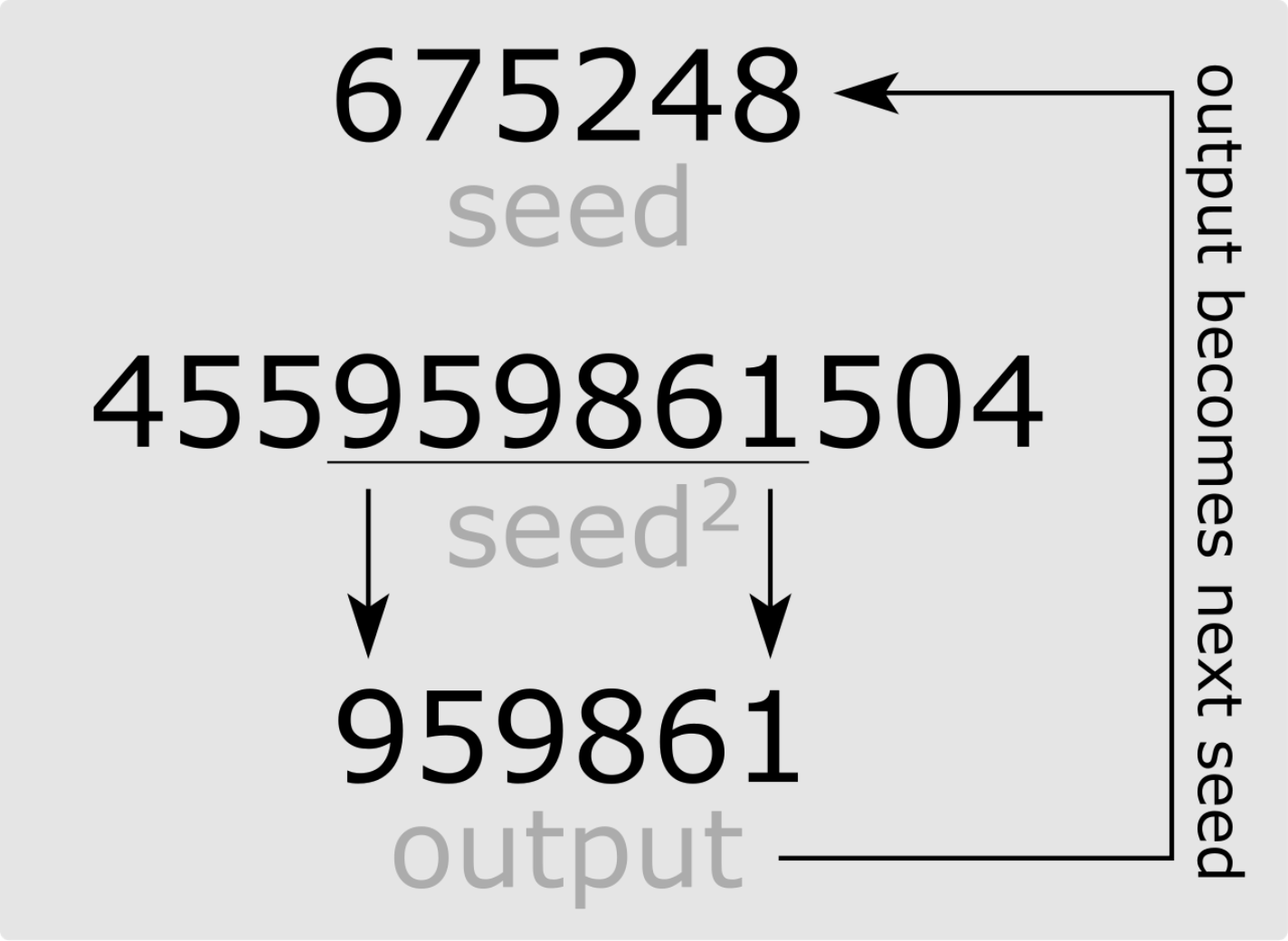
Possible definitions:

"There must not exist a randomized polynomial time algorithm that can differentiate between truly random and generated sequences with *non-negligible advantage*."

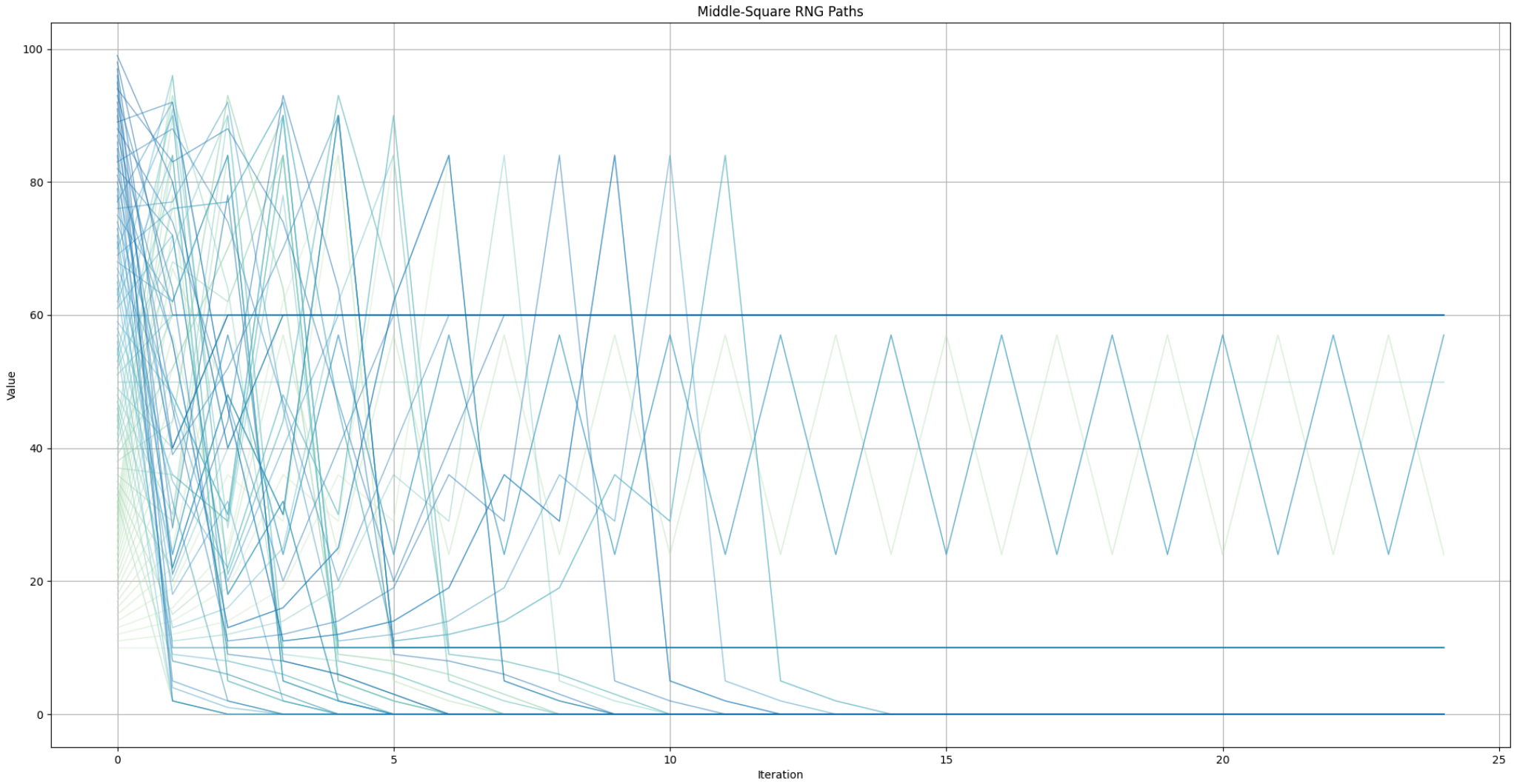
"There must not exists a polynomial time algorithm that can guess the next bit given the previous bits."

2. The classics

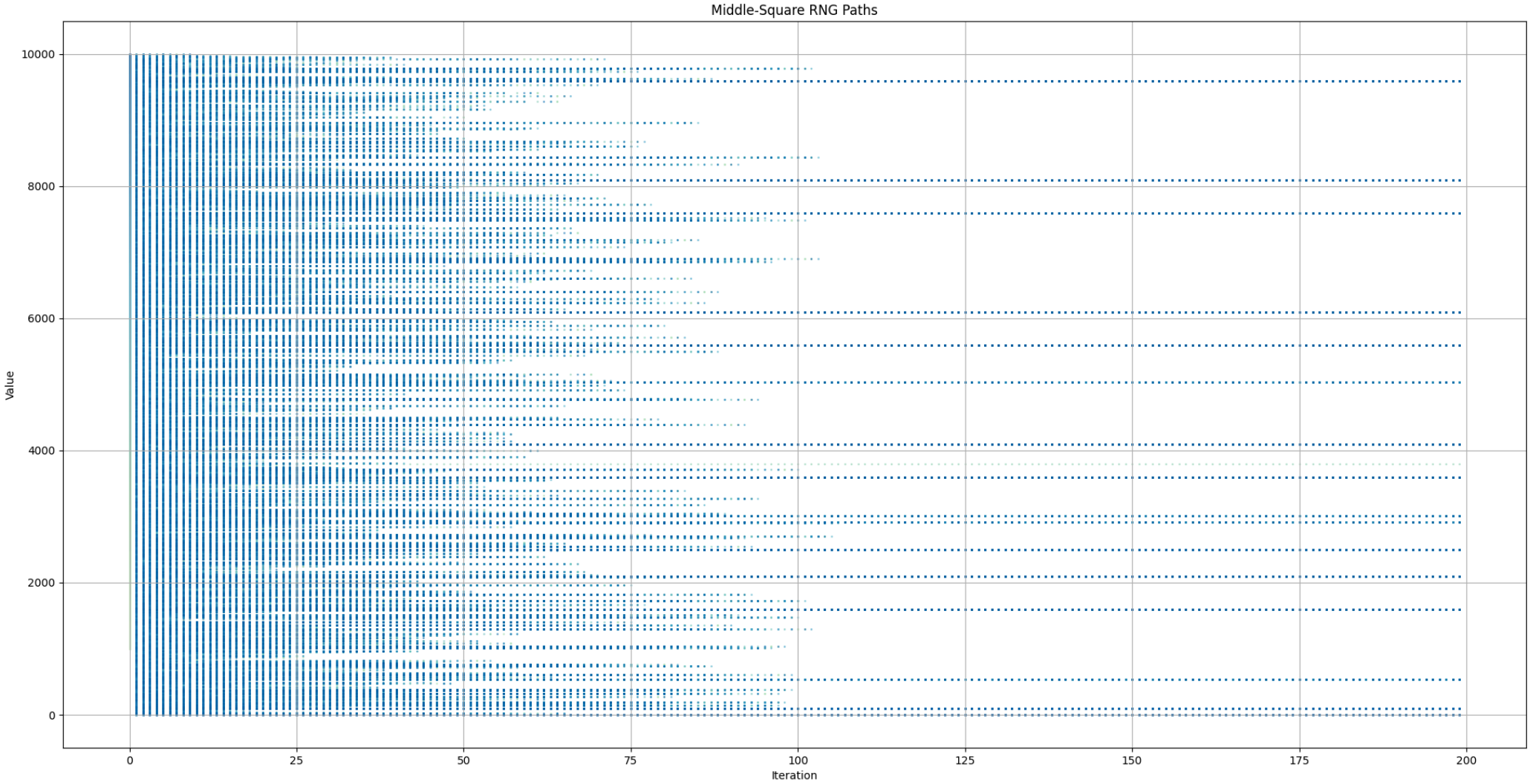
2.1 Middle square method (1946)



2.2 Problem with middle square



2.2 Problem with middle square



2.2 Problem with middle square

- After a couple iterations it is no longer random.
  For $n = 2$ after ≈ 10 iterations, for $n = 4$ after ≈ 100 iterations it no longer produces meaningful numbers.

2.2 Problem with middle square

- After a couple iterations it is no longer random.
  For $n = 2$ after ≈ 10 iterations, for $n = 4$ after ≈ 100 iterations it no longer produces meaningful numbers.

- Often enters a short cycle.
  For $n = 2$ there are 4 fixed points $0, 10, 50, 60$ and a two number cycle $24 \leftrightarrow 57$.

2.2 Problem with middle square

- After a couple iterations it is no longer random.
  For $n = 2$ after ≈ 10 iterations, for $n = 4$ after ≈ 100 iterations it no longer produces meaningful numbers.
- Often enters a short cycle.
  For $n = 2$ there are 4 fixed points $0, 10, 50, 60$ and a two number cycle $24 \leftrightarrow 57$.
- If middle digits are $00\ldots0$ it no longer produces meaningful numbers.

2.2 Problem with middle square

- After a couple iterations it is no longer random.
  For $n = 2$ after ≈ 10 iterations, for $n = 4$ after ≈ 100 iterations it no longer produces meaningful numbers.
- Often enters a short cycle.
  For $n = 2$ there are 4 fixed points $0, 10, 50, 60$ and a two number cycle $24 \leftrightarrow 57$.
- If middle digits are $00\ldots0$ it no longer produces meaningful numbers.

> "Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin."
>
> — John von Neumann

2.3 Linear congruential generator (1951)

$$X_i = aX_{i-1} + b \pmod{m}$$

2.3 Linear congruential generator (1951)

$$X_i = aX_{i-1} + b \pmod{m}$$

RANDU (1960-1970)

$$X_i = 65539 \cdot X_{i-1} \pmod{2^{31}}$$

2.3 Linear congruential generator (1951)

$$X_i = aX_{i-1} + b \pmod{m}$$

RANDU (1960-1970)

$$X_i = 65539 \cdot X_{i-1} \pmod{2^{31}}$$

https://github.com/leonardo-toffalini/typsting/blob/main/kript/randu_planes.gif

Outputs fall into only 15 planes during the spectral test.

2.4 Park–Miller (1988)

$$X_i = 7^5 \cdot X_{i-1} \pmod{2^{31} - 1}$$

"Give me something I can understand, implement and port… it needn't be state-of-the-art, just make sure it's reasonably good and efficient."

— Park–Miller

2.4 Park–Miller (1988)

$$X_i = 7^5 \cdot X_{i-1} \pmod{2^{31} - 1}$$

> "Give me something I can understand, implement and port… it needn't be state-of-the-art, just make sure it's reasonably good and efficient."
>
> — Park–Miller

Problem with LCGs

It can be proved that the elements of LCG sequences can be guessed in polynomial time with polynomial many known elements of the sequence with a sufficiently complicated algorithm.

2.5 Shift register (1965)

$$a_k = f(a_{k-1}, a_{k-2}, ..., a_{k-n})$$

2.5 Shift register (1965)

$$a_k = f(a_{k-1}, a_{k-2}, ..., a_{k-n})$$

Linear shift register

$$f(x_0, ..., x_{n-1}) = b_0 x_0 + b_1 x_1 + ... + b_{n-1} x_{n-1} \qquad (b_i \in \{0, 1\})$$

2.5 Shift register (1965)

$$a_k = f(a_{k-1}, a_{k-2}, ..., a_{k-n})$$

Linear shift register

$$f(x_0, ..., x_{n-1}) = b_0 x_0 + b_1 x_1 + ... + b_{n-1} x_{n-1} \qquad (b_i \in \{0, 1\})$$

Xorshift

$$y_1 = x_n \oplus (x_n \ll 13)$$
$$y_2 = y_1 \oplus (y_1 \gg 17)$$
$$x_{n+1} = x_2 \oplus (x_2 \ll 5)$$

2.5 Shift register (1965)

$$a_k = f(a_{k-1}, a_{k-2}, ..., a_{k-n})$$

Linear shift register

$$f(x_0, ..., x_{n-1}) = b_0 x_0 + b_1 x_1 + ... + b_{n-1} x_{n-1} \qquad (b_i \in \{0, 1\})$$

Xorshift

$$\begin{aligned} y_1 &= x_n \oplus (x_n \ll 13) \\ y_2 &= y_1 \oplus (y_1 \gg 17) \\ x_{n+1} &= x_2 \oplus (x_2 \ll 5) \end{aligned}$$

Alternatively, in $\mathbb{F}_2^{32}$

$$x_{n+1} = (1 \oplus 2^5)(1 \oplus 2^{32-17})(1 \oplus 2^{13}) x_n$$

2.6 Problem with shift registers

The coefficients can be recovered by solving the following linear system.

$$
\begin{aligned}
b_0 a_0 + b_1 a_1 + \ldots + b_{n-1} a_{n-1} &= a_n \\
b_0 a_1 + b_1 a_2 + \ldots + b_{n-1} a_n &= a_{n+1} \\
\vdots \qquad\qquad \vdots & \\
b_0 a_{n-1} + b_1 a_n + \ldots + b_{n-1} a_{2-2} &= a_{2n-1}
\end{aligned}
$$

2.7 Square root generator

Example:

$$\sqrt{5} = 10.\overbrace{0011100011011}^{f(5)}...$$

For a seed a we can define it as

$$f(a) = \sqrt{a} - \lfloor \sqrt{a} \rfloor$$

2.7 Square root generator

Example:

$$\sqrt{5} = 10.\overbrace{0011100011011}^{f(5)}\ldots$$

For a seed a we can define it as

$$f(a) = \sqrt{a} - \lfloor \sqrt{a} \rfloor$$

Problem with square root generator

Seems random but is still *breakable* with a sufficiently complicated number theoretic approach.

2.8 Mersenne twister (1997)

- Your programming language probably uses this one.

2.8 Mersenne twister (1997)

- Your programming language probably uses this one.
- Still just a shift register, but with a few tricks.

2.8 Mersenne twister (1997)

- Your programming language probably uses this one.
- Still just a shift register, but with a few tricks.
- Keep a state from which we recursively generate.

2.8 Mersenne twister (1997)

- Your programming language probably uses this one.
- Still just a shift register, but with a few tricks.
- Keep a state from which we recursively generate.
- The recurrence is offset.

2.8 Mersenne twister (1997)

- Your programming language probably uses this one.
- Still just a shift register, but with a few tricks.
- Keep a state from which we recursively generate.
- The recurrence is offset.
- Before returning the output temper with it.

2.8 Mersenne twister (1997)

$$x_{k+n} := x_{k+m} \oplus \left(\left(x_k^u \mid x_{k+1}^l \right) A \right) \qquad (k = 0, 1, 2, \ldots)$$

2.8 Mersenne twister (1997)

$$x_{k+n} := x_{k+m} \oplus \left(\left(x_k^u \mid x_{k+1}^l \right) A \right) \qquad (k = 0, 1, 2, \ldots)$$

$$xA = \begin{cases} x \gg 1 & x_0 = 0 \\ (x \gg 1) \oplus a & x_0 = 1 \end{cases} \qquad (a = 9908\mathrm{B0DF}_{16})$$

2.8 Mersenne twister (1997)

$$x_{k+n} := x_{k+m} \oplus \left(\left(x_k^u \mid x_{k+1}^l \right) A \right) \qquad (k = 0, 1, 2, \ldots)$$

$$xA = \begin{cases} x \gg 1 & x_0 = 0 \\ (x \gg 1) \oplus a & x_0 = 1 \end{cases} \qquad (a = 9908\mathrm{B0DF}_{16})$$

Then temper with the output

$$\begin{aligned} y &= x \oplus ((x \gg u) \& d) & (u, d) &= (11, \mathrm{FFFFFFFF}_{16}) \\ y &= y \oplus ((y \ll s) \& b) & (s, b) &= (7, \mathrm{9D2C5680}_{16}) \\ y &= y \oplus ((y \ll t) \& c) & (t, c) &= (15, \mathrm{EFC60000}_{16}) \\ z &= y \oplus (y \gg l) & (l &= 18) \end{aligned}$$

Output z.

3. Formalism

3.1 Definitions

Definition 3.1.1 A function $f : \mathbb{Z}^+ \to \mathbb{R}$ is negligible if for all fixed k $\lim_{n\to\infty} n^k f(n) \to 0$.

$f(n) = \mathrm{NEGL}(n)$

Definition 3.1.2 A function $G : \{0,1\}^* \to \{0,1\}^*$ is a generator if $|G(x)|$ only depends on $|x|$ and $|x| < |G(X)| < |x|^c$ for some constant c.

Definition 3.1.3 Let $\mathcal{A}$ be a randomized polynomial time algorithm that for each $z \in \{0,1\}^*$ input it outputs a bit $\mathcal{A}(z) \in \{0,1\}$ meaning the input was random (1) or not (0). $\mathcal{A}$ is called a test.

3.1 Definitions

Definition 3.1.4 For a fixed $n \geq 1$ chose uniformly at random x from $\{0,1\}^n$ and y from $\{0,1\}^N$ where $N = |G(x)|$. With equal probability give either $G(x)$ or y as input to $\mathcal{A}$. We say that $\mathcal{A}$ was a successful test if it correctly determined whether it had a random input or not.

Definition 3.1.5 We say that a generator G is secure if for all randomized polynomial time algorithms $\mathcal{A}$ the probability of $\mathcal{A}$ being successful is $\frac{1}{2} + \mathrm{NEGL}(n)$.

Remark

In essence, G passes all *meaningful* tests, that is, the best test is to guess at random.

3.2 Equivalent definition of secure generator

Definition 3.2.1 A generator $g(x) = G_1 G_2 ... G_N$ is said to be unpredictable if

$$\mathbb{P}(\mathcal{B}(n, G_1 ... G_i) = G_{i+1}) = \frac{1}{2} + \mathrm{NEGL}(n) \quad \forall i \underset{R}{\in} \{1, ..., N\}$$

where $x \underset{R}{\in} \{0,1\}^n$.

Theorem 3.2.2 (Yao) A generator G is secure if and only if it is unpredictable.

Remark

Clearly, if we reverse a secure generators output, then we get another secure generator, since any algorithm that could differentiate the original output can differentiate the reversed output by first reversing it themselves.

3.3 How hard could it be?

Proposition 3.3.1 If $\mathbf{P} = \mathbf{NP}$, then there is no secure generator.

Proof. Fix a generator G.

Define $L := \{y : \exists x \in \{0,1\}^* \text{ such that } \; y = G(x)\}$. Clearly $L \in \mathbf{NP}$, since x is a polynomial proof for $y \in L$.

By $\mathbf{P} = \mathbf{NP} \implies L \in \mathbf{P} \implies \exists \mathcal{A}$ PTA that decides $x \overset{?}{\in} L$.

From this we have that $\mathcal{A}$ is always successful in recognizing $G(x)$, that is G is not secure.

$\square$

Remark

If you find a secure generator you can claim your \$1M, since you have proven $\mathbf{P} \neq \mathbf{NP}$.

3.4 One-way functions

Definition 3.4.1 We say that $f : \{0,1\}^* \to \{0,1\}^*$ is a one-way function, if

- $\exists c \geq 1$ such that $|x|^{\frac{1}{c}} < |f(x)| < |x|^c$
- $f(x)$ is polynomial time computable
- $\forall \mathcal{A} : \{0,1\}^* \to \{0,1\}^*$ RPA and $y \underset{R}{\in} \{0,1\}^*$ the following holds:

$$\mathbb{P}(f(\mathcal{A}(f(y))) = f(y)) = \mathrm{NEGL}(n).$$

Remark

Since f must not be invertible we could not write $\mathcal{A}(f(y)) = y$.

3.4 One-way functions

Definition 3.4.2 We say that f is a one-way permutation if it is a bijection and $|f(x)| = |x| \quad \forall x$.

Theorem 3.4.3 (Goldreich–Levin) If f is a one-way permutation, then we can construct a secure generator from f.

3.5 Construction of Goldreich–Levin generator

Choose a seed $(x, p) \underset{R}{\in} \{0,1\}^n \times \{0,1\}^n$.

3.5 Construction of Goldreich–Levin generator

Choose a seed $(x, p) \underset{R}{\in} \{0,1\}^n \times \{0,1\}^n$.

Compute $y^{(t)} = f^t(x) + f(f^{t-1}(x))$ for all $t = 1, \ldots, N$.

3.5 Construction of Goldreich–Levin generator

Choose a seed $(x, p) \underset{R}{\in} \{0,1\}^n \times \{0,1\}^n$.

Compute $y^{(t)} = f^t(x) + f(f^{t-1}(x))$ for all $t = 1, \ldots, N$.

Output $g(x, p) = G_1 \cdot G_N$, where $G_t = p \cdot y^{(t)}$.

3.5 Construction of Goldreich–Levin generator

Choose a seed $(x, p) \underset{R}{\in} \{0,1\}^n \times \{0,1\}^n$.

Compute $y^{(t)} = f^t(x) + f(f^{t-1}(x))$ for all $t = 1, \ldots, N$.

Output $g(x, p) = G_1 \cdot G_N$, where $G_t = p \cdot y^{(t)}$.

Where

$$(a_i)_{i=1}^n \cdot (b_i)_{i=1}^n := \bigoplus_{i=1}^n a_i b_i = a_1 b_1 \oplus a_2 b_2 \oplus \ldots \oplus a_n b_n.$$

3.5 Construction of Goldreich–Levin generator

Choose a seed $(x, p) \underset{R}{\in} \{0,1\}^n \times \{0,1\}^n$.

Compute $y^{(t)} = f^t(x) + f(f^{t-1}(x))$ for all $t = 1, \ldots, N$.

Output $g(x, p) = G_1 \cdot G_N$, where $G_t = p \cdot y^{(t)}$.

Where

$$(a_i)_{i=1}^n \cdot (b_i)_{i=1}^n := \bigoplus_{i=1}^n a_i b_i = a_1 b_1 \oplus a_2 b_2 \oplus \ldots \oplus a_n b_n.$$

Why this should work

Let $y = f^i(x)$ and $z = f^{i+1}(x)$, since f is a one-way function, no one can compute y from z with non-negligible advantage.

3.5 Construction of Goldreich–Levin generator

Choose a seed $(x, p) \underset{R}{\in} \{0,1\}^n \times \{0,1\}^n$.

Compute $y^{(t)} = f^t(x) + f(f^{t-1}(x))$ for all $t = 1, \ldots, N$.

Output $g(x, p) = G_1 \cdot G_N$, where $G_t = p \cdot y^{(t)}$.

Where

$$(a_i)_{i=1}^n \cdot (b_i)_{i=1}^n := \bigoplus_{i=1}^n a_i b_i = a_1 b_1 \oplus a_2 b_2 \oplus \ldots \oplus a_n b_n.$$

Why this should work

Let $y = f^i(x)$ and $z = f^{i+1}(x)$, since f is a one-way function, no one can compute y from z with non-negligible advantage.

In essence, even if someone knows p and z they cannot compute

$$G_i = p \cdot f^{-1}(z).$$

4. One-way function candidates

4.1 Factorization

Let p and q be two n long prime numbers.

Let $f(n, p, q) = pq$.

Given $f(n, p, q)$ try to guess p and q.

4.1 Factorization

Let p and q be two n long prime numbers.

Let $f(n, p, q) = pq$.

Given $f(n, p, q)$ try to guess p and q.

> **Remark**
>
> Shor's algorithm shows that this function is reversible in polynomial time with a quantum computer.

4.2 Discrete logarithm

Given a prime p and a primitive root g and $k < p$, let $y = g^k \bmod p$ the output is (p, g, y).

Try to guess k, that is the discrete logarithm of y modulo p.

4.2 Discrete logarithm

Given a prime p and a primitive root g and $k < p$, let $y = g^k \bmod p$ the output is (p, g, y).

Try to guess k, that is the discrete logarithm of y modulo p.

> **Remark**
>
> Shor's algorithm solves this problem too.

4.3 Discrete square root

Given m and $x < m$ the output is m and $y = x^2 \bmod m$.

Try to find x such that $x^2 \equiv y \pmod{m}$

5. Beyond random bits

5.1 Uniform distribution

Generate a random number $X \sim U(0,1)$

Generate an n long binary sequence $(a_i)_{i=1}^n$

$$X = \sum_{i=1}^{n} \frac{a_i}{2^i}$$

$$X = \overline{0.a_1 a_2 a_3 \ldots a_n}_2$$

This has precision 2^{-n}.

5.2 Box–Muller transform

Suppose you have $U_1, U_2 \sim U(0,1)$.

Let

$$Z_1 = \sqrt{-2\log U_1}\cos(2\pi U_2)$$
$$Z_2 = \sqrt{-2\log U_1}\sin(2\pi U_2)$$

Then Z_1, Z_2 are independent standard normal variables.

https://upload.wikimedia.org/wikipedia/commons/1/1f/Box-Muller_transform_visualisation.svg